MCP + RAG for 1C

Local-first search over BSL business logic and configuration metadata: hybrid BM25 + cosine via RRF, and an honest evaluation

Sergey Matyukhin

github.com/Smaturus/1c-mcp-rag-search

MCP Python SDK · Chroma · multilingual-e5-small · BM25 · RRF · flashrank

English talk · 10 minutes + Q&A

The problem, and what I built

Problem

- A 1C configuration dump is thousands of .bsl modules plus a multi-megabyte metadata report
- grep matches text, not meaning; an exact handler name and an intent question need different tools
- Hosted search stacks mean deployment work and data leaving the machine
- An LLM agent needs a tool whose quality is measured, not assumed

What I built

- A local MCP server over stdio: 9 tools, including an explainable router
- Fully local retrieval: e5-small embeddings plus Chroma, CPU only, no external APIs
- Hybrid search: cosine and a global BM25 index, fused with Reciprocal Rank Fusion
- Measured: gold-set v2, 70 queries, file-level relevance, 4 independent runs

The project in numbers

179,586
chunks in the index

0.339
nDCG@10, vector (leader)

68/68
router regression cases

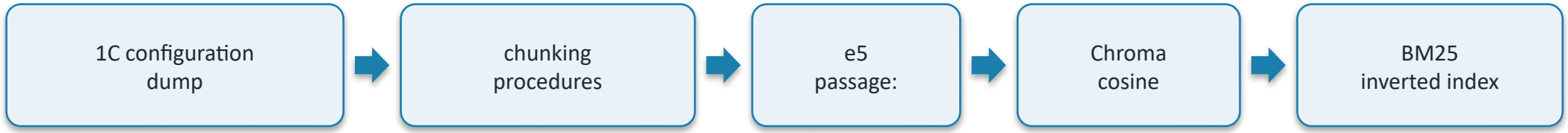
99.7 %
metadata BM25 Hit@3

Stack

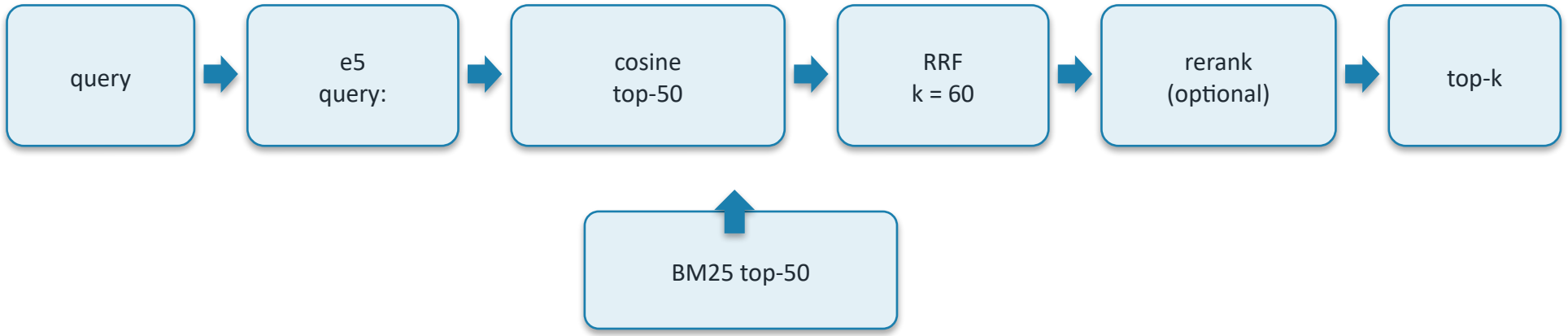
- Embeddings: intfloat/multilingual-e5-small, 384 dimensions, 512-token limit, query:/passage: prefixes
- Storage: Chroma with HNSW cosine; lexical side: a custom BM25 inverted index over all chunks
- Fusion: Reciprocal Rank Fusion, k = 60; optional flashrank cross-encoder, off by default
- Interface: MCP over stdio, 9 tools; evaluation: 70 queries x 4 configurations, 4 runs each
- Engineering: Python 3.11/3.12, 72 unit tests, ruff clean, mypy on 22 modules, complexity gate 143 blocks (avg 4.37, A); 40-60 ms/query

Architecture: two loops

Offline index build



Online query path (MCP over stdio)



Top: the offline index build. Bottom: the online query path. RRF fuses two independent rankings; the cross-encoder is an optional extra stage. Everything runs locally: no external APIs, no data leaves the machine.

Corpus and chunking

Procedure-level chunking

- 154,560 code chunks: one procedure or function per chunk (ADR 0007)
- Before (symbolic 1200/200 windows): procedures cut mid-body, 32.1% of chunks over the 512-token limit
- 25,026 metadata chunks parsed from the configuration XML dump
- Total: 179,586 chunks in the index
- Current gates: 99.6% code coverage; 0.3% over 512 tokens; tokens p50/p99/max = 179 / 472 / 554

Metadata from XML

- 25,026 chunks over 5,911 objects: attributes, types and form names
- BM25 Hit@3 99.7% against 93.0% for the old text-report source (ADR 0008)
- Structured source keeps field names and types inside the chunk text

93.0% → 99.7%

BM25 Hit@3 on metadata, text report → XML

Parsed once into JSONL, then indexed like code.

MCP server: tools for an agent

Tool	What it returns / when to use it
codesearch(query, limit)	cosine baseline over BSL code modules
metadatasearch(query, limit)	cosine baseline over configuration metadata
search(query, top_k, mode, collection)	unified entry: vector | hybrid | hybrid_rerank | pearson
evaluate_retrieval(query, k, ...)	Hit@k, MRR, nDCG and a null-tail check for one query
search_1c_metadata(query, limit)	BM25: exact names, handlers, calls, "who writes to this register"
search_knowledge_base(query, limit)	vector: "how to", "why it fails", instructions
search_hybrid(query, limit)	RRF fusion for mixed queries; the safe default
route_query(query)	explains route, tool, confidence and reason
index_stats()	diagnostic: which index is served, collection counts, model, default mode

How it connects

- stdio transport: the client launches the server, there is no service to deploy
- Routing is code with tests, not instructions inside a prompt: 68/68 on the dev regression set
- Blind Part-3 set (20 cases, hash fixed): 75% before patches, published as below the 80% gate; 20/20 after, but that set is dev now - a new hold-out is needed

Hybrid retrieval: Reciprocal Rank Fusion

$$score(d) = \Sigma\ 1 / (k + rank_i(d)),\quad k = 60$$

Why RRF

- cosine and BM25 produce scores on incomparable scales
- RRF uses ranks only, so there is nothing to calibrate
- BM25 catches exact names, cosine catches meaning; fusion keeps both
- k = 60 damps the first positions so one source cannot dominate

What it bought us (nDCG@10)

- Old corpus: hybrid 0.209, vector 0.176 - hybrid was the leader
- Current corpus: vector 0.339, hybrid 0.301
- A weighted Pearson variant reaches 0.337, but needs tuned weights
- Latency stays at 40-60 ms per query on CPU

Example: query "ПередЗаписью". BM25 lifts modules that contain the exact handler name, cosine adds related meaning, and RRF merges the two rankings without a weight search.

Negative result: the cross-encoder reranker

What I built

- flashrank 0.2.10 with an ONNX cross-encoder on CPU, no PyTorch
- ms-marco-MultiBERT-L-12, max_length = 128
- Input: top-20 RRF candidates; a passage is the file path plus its code
- Lazy model load, cache on disk, one extra stage after fusion

What it did to quality

- Old corpus: 0.209 → 0.069
- Current corpus: 0.117 against 0.301 for plain hybrid
- Latency ~2.4 s per query on CPU
- Hypothesis: trained on web passages; BSL is long, repetitive, identifier-heavy code
- Decision: kept as an option, not the default (ADR 0002)

The losing configuration stays in the report. Nothing was tuned until the numbers looked good.

How quality was measured

Gold-set v2

- 70 natural-language queries: 30 curated plus 40 generated over configuration objects
- Three query types: exact, semantic, trap
- Relevance is scored at the source-file level, not per chunk
- Duplicate chunks from one file are collapsed before scoring (ADR 0004)
- Metrics: Hit@1, MRR@10, nDCG@10, Recall@10; four independent runs, a fresh process each

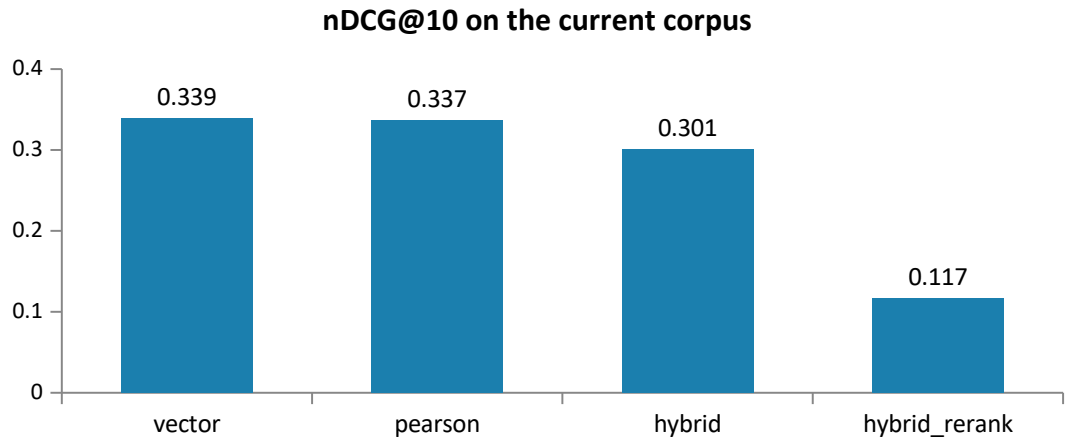
Why this way

- Chunk-level ground truth scored a correct file as a miss: false zeros
- File-level relevance stays stable when the chunking changes
- Recall@k is normalised by min(|relevant|, k)
- std of 0.023 for vector means the result is reproducible, not a lucky run
- Each run reloads the index: the numbers are means, not a best-of selection

Every number in this talk is a mean of four independent runs, reported together with its standard deviation.

Results: gold-set v2, 70 queries, k = 10

Configuration	Hit@1	MRR@10	nDCG@10	Recall@10
vector	0.150 ± 0.007	0.268 ± 0.019	0.339 ± 0.023	0.523 ± 0.030
pearson	0.179 ± 0.021	0.270 ± 0.024	0.337 ± 0.030	0.511 ± 0.048
hybrid (RRF)	0.129 ± 0.010	0.240 ± 0.013	0.301 ± 0.017	0.482 ± 0.034
hybrid + rerank	0.086	0.102	0.117	0.157



All CPU modes: 40-60 ms per query. Cross-encoder rerank: ~2.4 s.

Validation: v2 and independent v3 hold-out

v2: Δ +0.037, CI [-0.015; +0.086], p = 0.154
v3 (71 code queries, 3 runs): vector 0.205 ± 0.014 / hybrid 0.183 ± 0.005
v3: Δ +0.023, CI [-0.019; +0.061], p = 0.275 — not significant
Both not significant: default hybrid unchanged, ADR 0009 stays proposed.

2×2 ablation (nDCG@10, vector / hybrid)

2×2 nDCG@10	vector	hybrid
old export · symbolic	0.176	0.209
old export · procedural	—	—
current export · symbolic	0.165 ± 0.013	0.205 ± 0.007
current export · procedural	0.339 ± 0.023	0.301 ± 0.017

Chunking explains the flip: hybrid leads under symbolic chunking, vector leads under procedural. Export effect at symbolic chunking: −0.012 / −0.004, inside run noise.

Interaction cell missing (old dump gone); additivity assumed.

Decisions and limitations

Decision log (ADR)

- 0001: RRF over weighted Pearson - better ranking, no weight calibration
- 0002: reranker stays optional - it hurts quality on BSL
- 0003: file-level relevance - stable, reproducible metrics
- 0004: deduplicated metrics - one file counted once
- 0005: gold-set v2 - natural-language queries, not identifier templates
- 0006: routing as tested code - tool choice is explainable
- 0007: procedure chunking - the 512-token overflow is gone
- 0008: XML metadata - BM25 Hit@3 93.0% → 99.7%
- 0009: corpus change - leader moved from hybrid to vector (proposed)

Limitations

- HNSW is rebuilt on start: ~60-70 s cold start, a workaround for a known Chroma issue
- Gold-set is partly synthetic; absolute metrics are modest
- A single writer; no parallel indexing
- No incremental indexing: a full rebuild takes hours
- v3 hold-out: router 58/100; quoted-name lexical cases fall back to hybrid

ADR 0009 is deliberately still proposed: the default changes only after an independent sample confirms the shift.

What is done, and what comes next

What is done

- MCP server with 9 tools over a 179,586-chunk local index
- Retrieval: e5 cosine + global BM25 + RRF + an optional cross-encoder
- Router as a tested contract: 68/68 on the dev regression set
- Evaluation loop: gold-set v2, file-level metrics, 4 runs, mean ± std
- Engineering: 72 tests, ruff + mypy (22 modules) + complexity gate (avg 4.37 A), ADR log, CI green

What comes next

- Finish the 2x2 ablation, then decide the default mode (ADR 0009)
- A new hold-out for the router; incremental indexing; get_metadata and find_callers
- Code-oriented embeddings; a code reranker; 100+ manually labelled queries

github.com/Smaturus/1c-mcp-rag-search · smaturus@gmail.com